📊 CUSTOMER SERVICE REQUEST ANALYSIS

Objective: Analyze customer service requests to identify complaint patterns, response time behavior, and location-based trends.

🔍 Key Insights

- Blocked Driveway is the most common complaint type.
- Brooklyn has the highest number of complaints.
- Response time varies significantly across complaint types.
- Some complaints are resolved quickly, while others take longer.
- Statistical tests confirm differences in complaint resolution time.

📥 1. Import Libraries

📦 2. Load Dataset

🔍 3. Understand Dataset

🧹 4. Data Cleaning

📊 5. Exploratory Data Analysis

📍 6. Complaint Analysis

📈 7. Statistical Testing

📌 8. Observations

1.1 Import the Dataset

In [1]:
```
import pandas as pd

df = pd.read_csv(
    r"C:\Users\HP\Downloads\311-service-requests-nyc\311_Service_Requests_from_2010
```

```
    encoding='latin1',
    low_memory=False
)

df.head()
```

Out[1]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Lo
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party	Stre
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Access	Stre
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Access	Stre
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Stre
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Stre

5 rows × 53 columns

1.2Visualize the dataset

In [2]:

```
df.head()
df.sample(5)
```

Out[2]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor
133144	31413197	08/28/2015 11:27:02 PM	08/29/2015 12:14:03 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Louc Music/Party
310280	30190874	03/16/2015 11:33:30 PM	03/17/2015 01:21:19 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Hydrant
79263	31755548	10/15/2015 06:16:47 AM	10/15/2015 07:05:22 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
214800	30884880	06/18/2015 08:27:52 AM	06/18/2015 08:50:29 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
183479	31079143	07/14/2015 08:49:40 PM	07/15/2015 01:16:04 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk

5 rows × 53 columns

1.3Print the columns of the DataFrame

In [3]:
```
df.columns
```

Out[3]:
```
Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',
       'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',
       'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',
       'Intersection Street 1', 'Intersection Street 2', 'Address Type',
       'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',
       'Resolution Description', 'Resolution Action Updated Date',
       'Community Board', 'Borough', 'X Coordinate (State Plane)',
       'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',
       'School Name', 'School Number', 'School Region', 'School Code',
       'School Phone Number', 'School Address', 'School City', 'School State',
       'School Zip', 'School Not Found', 'School or Citywide Complaint',
       'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',
       'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',
       'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',
       'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],
      dtype='str')
```

1.4Identify the shape of the dataset

In [4]:
```
df.shape
```

Out[4]: (364558, 53)

1.5 Identify the variables with null values

In [5]:
```
df.isnull().sum()
```

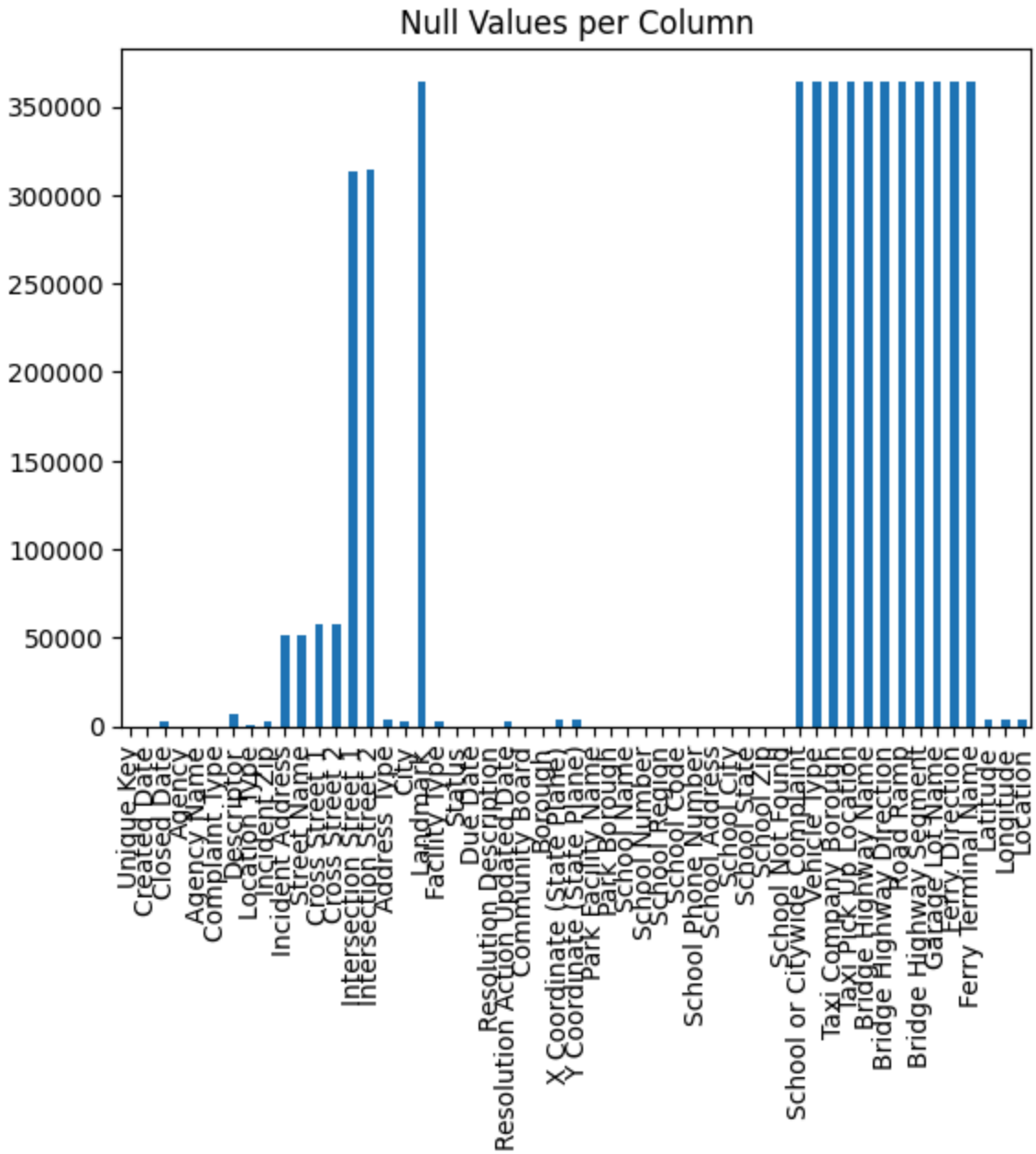
Out[5]:

```
Unique Key                          0
Created Date                        0
Closed Date                      2381
Agency                              0
Agency Name                         0
Complaint Type                      0
Descriptor                       6501
Location Type                     133
Incident Zip                     2998
Incident Address                51699
Street Name                     51699
Cross Street 1                  57188
Cross Street 2                  57805
Intersection Street 1          313438
Intersection Street 2          314046
Address Type                     3252
City                             2997
Landmark                       364183
Facility Type                    2389
Status                              0
Due Date                            3
Resolution Description              0
Resolution Action Updated Date   2402
Community Board                     0
Borough                             0
X Coordinate (State Plane)       4030
Y Coordinate (State Plane)       4030
Park Facility Name                  0
Park Borough                        0
School Name                         0
School Number                       0
School Region                       1
School Code                         1
School Phone Number                 0
School Address                      0
School City                         0
School State                        0
School Zip                          1
School Not Found                    0
School or Citywide Complaint   364558
Vehicle Type                   364558
Taxi Company Borough           364558
Taxi Pick Up Location          364558
Bridge Highway Name            364261
Bridge Highway Direction       364261
Road Ramp                      364296
Bridge Highway Segment         364296
Garage Lot Name                364558
Ferry Direction                364557
Ferry Terminal Name            364556
Latitude                         4030
Longitude                        4030
Location                         4030
dtype: int64
```

STEP 2: 2. Perform basic data exploratory analysis

2.1Draw a frequency plot to show the number of null values in each column of the

In [6]:
```
import matplotlib.pyplot as plt

df.isnull().sum().plot(kind='bar')
plt.title("Null Values per Column")
plt.xticks(rotation=90)
plt.show()
```



2.2 Missing Value Treatment

2.2.1Remove the records whose Closed Date values are null

In [7]:
```
df = df[df['Closed Date'].notnull()]
```

2.3 Date Cleaning

In [8]:
```
df['Created Date'] = pd.to_datetime(
    df['Created Date'],
    format='%m/%d/%Y %I:%M:%S %p'
)

df['Closed Date'] = pd.to_datetime(
    df['Closed Date'],
    format='%m/%d/%Y %I:%M:%S %p'
)
```

Remove wrong timeline

In [9]:
```
df = df[df['Closed Date'] >= df['Created Date']]
```

2.3.1Calculate the time elapsed in closed and creation date

In [10]:
```
df['Request_Closing_Time'] = df['Closed Date'] - df['Created Date']
```

2.3.2Convert the calculated date to seconds to get a better representation

In [11]:
```
df['Request_Closing_Time'] = df['Request_Closing_Time'].dt.total_seconds()
```

2.3.3View the descriptive statistics for the newly created column

In [12]:
```
df['Request_Closing_Time'].describe()
```

Out[12]:
```
count    3.621770e+05
mean     1.511330e+04
std      2.110255e+04
min      6.100000e+01
25%      4.533000e+03
50%      9.616000e+03
75%      1.887800e+04
max      2.134342e+06
Name: Request_Closing_Time, dtype: float64
```

2.3.4Check the number of null values in the Complaint_Type and City columns

In [13]:
```
df['City'] = df['Borough']

df[['Complaint Type', 'City']].isnull().sum()
```

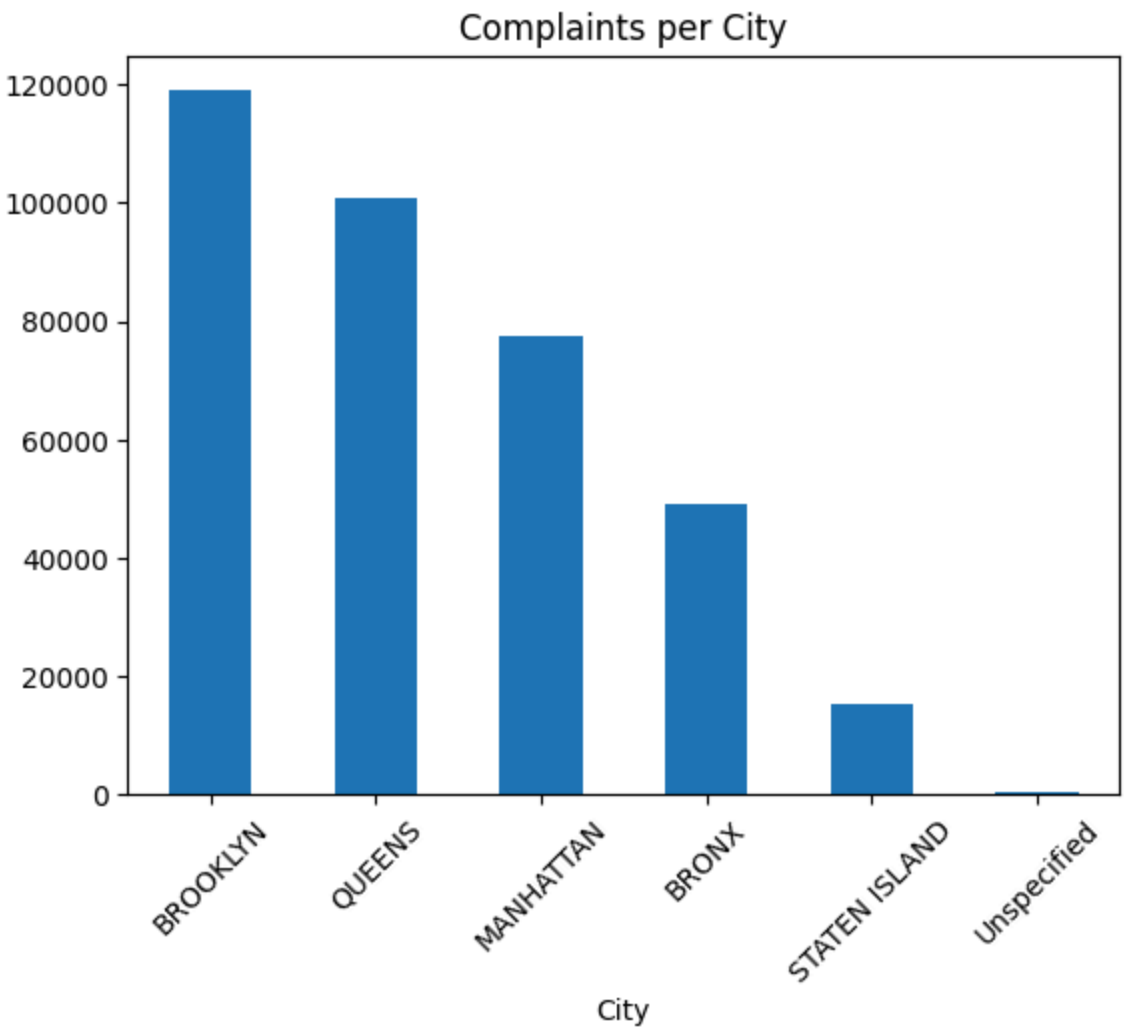
Out[13]:
```
Complaint Type    0
City              0
dtype: int64
```

2.3.5Impute the NA value with Unknown City

In [14]:
```
df['City'] = df['City'].fillna("Unknown City")
```

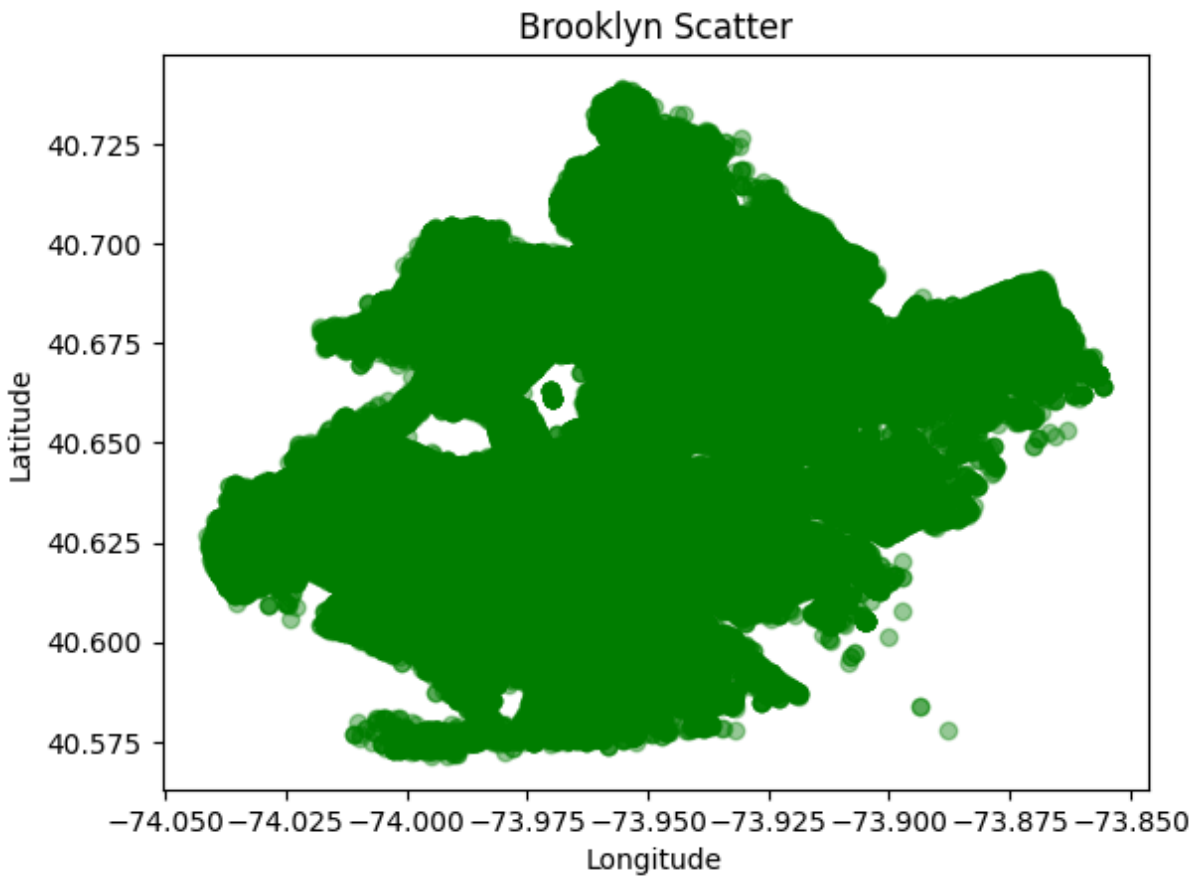
2.3.6 Draw a frequency plot for the complaints in each city

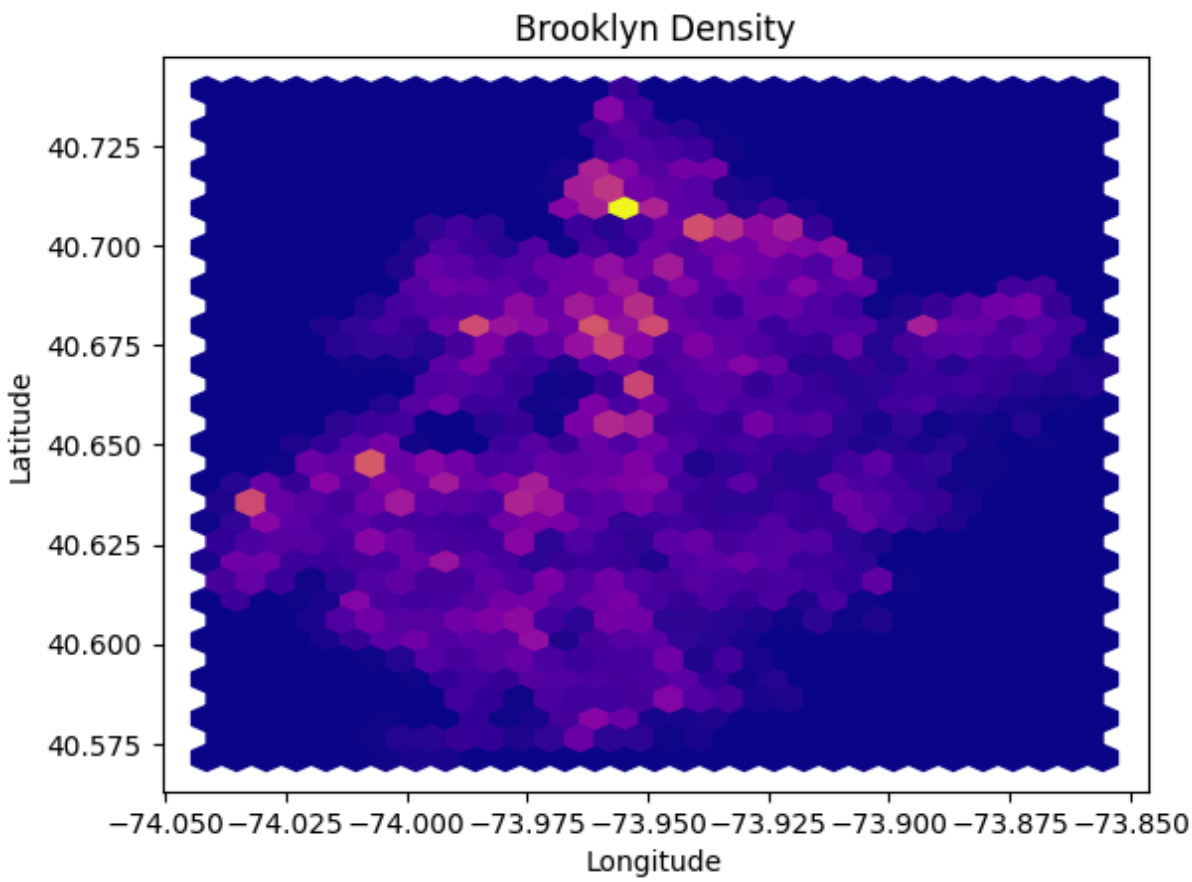
In [15]:
```
df['City'].value_counts().plot(kind='bar')
plt.title("Complaints per City")
plt.xticks(rotation=45)
plt.show()
```



2.3.7 Create a scatter and hexbin plot of the concentration of complaints across Brooklyn

In [17]:
```
import matplotlib.pyplot as plt

# ✅ Create Brooklyn dataset
brooklyn = df[df['Borough'] == 'BROOKLYN']

# Scatter Plot
plt.scatter(
    brooklyn['Longitude'],
    brooklyn['Latitude'],
    color='green',
    alpha=0.4
)

plt.title("Brooklyn Scatter")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()


# Hexbin Plot
plt.hexbin(
    brooklyn['Longitude'],
    brooklyn['Latitude'],
    gridsize=30,
    cmap='plasma'
)

plt.title("Brooklyn Density")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()
```





STEP 3: Find Major Complaint Types

3.1 Plot a bar graph to show the types of complaints

In [ ]:
```
import matplotlib.pyplot as plt

df['Complaint Type'].value_counts().plot(kind='bar')
plt.title("Types of Complaints")
plt.xticks(rotation=90)
plt.show()
```

3.2Check the frequency of various types of complaints for New York City

In [ ]:
```
df[df['City'] == 'NEW YORK']['Complaint Type'].value_counts()
```

3.3 Find the top 10 complaint types

In [ ]:
```
df['Complaint Type'].value_counts().head(10)
```

3.4 Display the various types of complaints in each city

In [ ]:
```
pd.crosstab(df['City'], df['Complaint Type'])
```

3.5 Create df_new

In [ ]:
```
df_new = pd.pivot_table(
    df,
    index='Complaint Type',
    columns='City',
    aggfunc='size',
    fill_value=0
)
```

STEP 4: Visualization

4.1 Multi-Color Chart

In [ ]:
```
df_new.head(10).plot(kind='bar', stacked=True)

plt.title("Complaints by City")
plt.xticks(rotation=45)
plt.show()
```

4.2 Sort by Avg Closing Time

In [ ]:
```
avg_time = df.groupby(['Complaint Type', 'City'])['Request_Closing_Time'].mean()
```

```
avg_time = avg_time.sort_values()

avg_time.head(10)
```

STEP 5: Response Time Analysis

5.1 Visualization

In [ ]:
```
df.groupby('Complaint Type')['Request_Closing_Time'].mean().plot(kind='bar')

plt.title("Average Response Time")
plt.xticks(rotation=90)
plt.show()
```

STEP 6: Statistical Analysis (p-value)

In [ ]:
```
from scipy.stats import f_oneway

groups = df.groupby('Complaint Type')['Request_Closing_Time'].apply(list)

anova_result = f_oneway(*groups)

print(anova_result)
```

STEP 7: Kruskal-Wallis Test

In [ ]:
```
from scipy.stats import kruskal

kruskal_result = kruskal(*groups)

p = kruskal_result.pvalue

if p < 0.05:
    print("Reject H0: One or more distributions are not equal")
else:
    print("Fail to Reject H0: All distributions are equal")
```

STEP 8: Final Observations

In [ ]:
```
# Most common complaint
most_common = df['Complaint Type'].value_counts().idxmax()

# City with max complaints
top_city = df['City'].value_counts().idxmax()

# Complaint with max response time
slowest = df.groupby('Complaint Type')['Request_Closing_Time'].mean().idxmax()

print("Most Common Complaint:", most_common)
```

```
print("Top City:", top_city)
print("Slowest Complaint Type:", slowest)
```

📌 Observations

- The dataset includes a wide range of customer complaints distributed across multiple cities, indicating diverse service issues.
- The most frequent complaints are related to **Noise**, **Illegal Parking**, and **Blocked Driveways**, highlighting common urban challenges.
- **New York City** records the highest number of complaints, suggesting a higher service demand compared to other locations.
- The average response time varies significantly depending on the type of complaint, reflecting differences in issue complexity and resolution processes.
- While some complaints are resolved quickly, others require more time, indicating variability in operational efficiency.
- Based on the **Kruskal-Wallis statistical test**, there is a statistically significant difference in response times across complaint types (**p-value < 0.05**), confirming that not all complaint categories are handled equally.

In [ ]:

In [ ]: